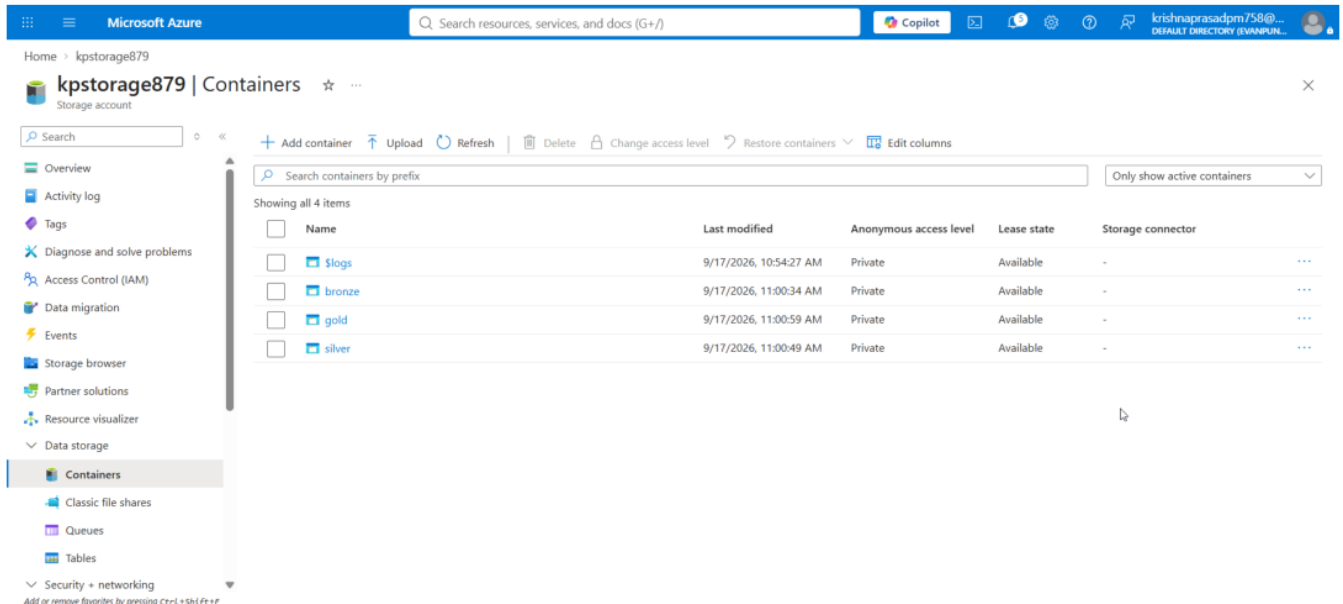


# Azure Databricks Data Engineering – ADLS Gen2 to Medallion Architecture

## Step 1: Review the ADLS Gen2 Storage Structure



Azure Storage account — Containers

### Step-by-step explanation

- Open the Azure Storage account **kpstorage879** and navigate to **Data storage** → **Containers**.
- Verify that the storage account contains the **\$logs**, **bronze**, **gold**, and **silver** containers.
- These containers establish the storage structure used by the data engineering workflow, with Bronze, Silver, and Gold representing progressively refined data layers.
- Before running Databricks processing, confirm that the required containers are available and accessible.

## Step 2: Verify the Bronze / Raw Data Layer

The screenshot shows the Microsoft Azure portal interface. The top navigation bar includes the Microsoft Azure logo, a search bar, and the user profile. The main content area displays the 'bronze' container within a storage account. The container's overview page shows a list of 10 items, all of which are CSV files. The files are listed with their names, last modified dates, access tiers, blob types, sizes, and lease states. The 'AdventureWorks\_Products.csv' file is highlighted with a mouse cursor.

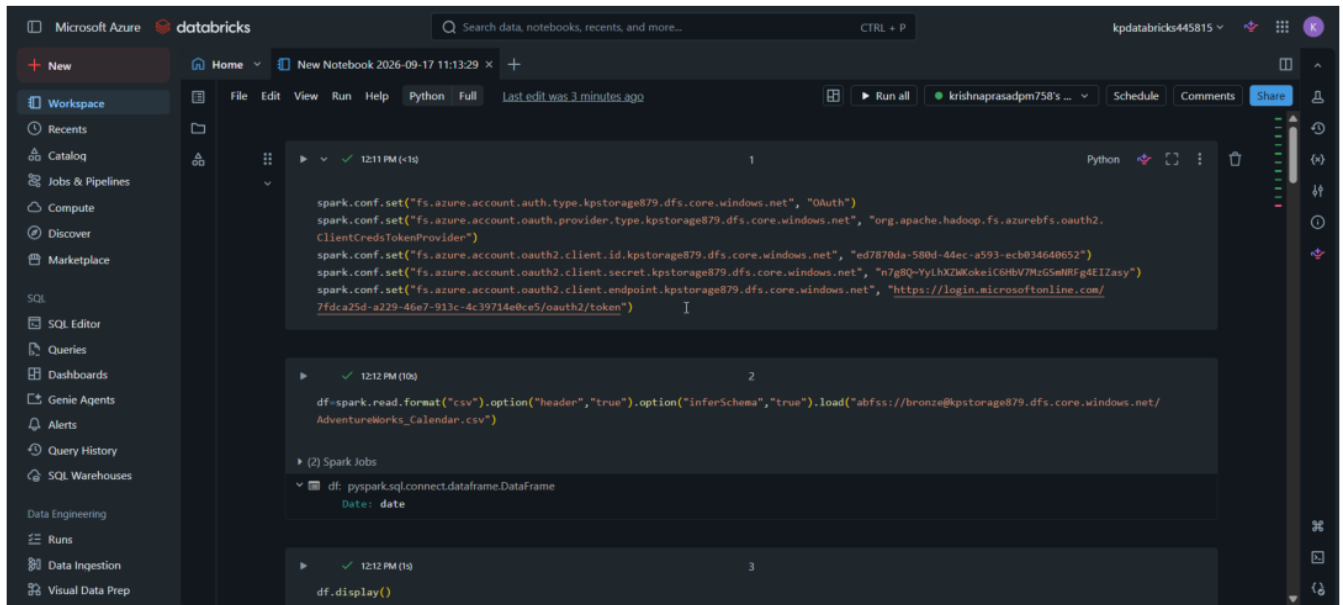
| Name                                     | Last modified          | Access tier    | Blob type  | Size       | Lease state |
|------------------------------------------|------------------------|----------------|------------|------------|-------------|
| AdventureWorks_Calendar.csv              | 9/17/2026, 11:02:16 AM | Hot (Inferred) | Block blob | 9.72 KiB   | Available   |
| AdventureWorks_Customers.csv             | 9/17/2026, 11:02:18 AM | Hot (Inferred) | Block blob | 1.87 MiB   | Available   |
| AdventureWorks_Product_Categories.csv    | 9/17/2026, 11:02:16 AM | Hot (Inferred) | Block blob | 83 B       | Available   |
| AdventureWorks_Product_Subcategories.csv | 9/17/2026, 11:02:16 AM | Hot (Inferred) | Block blob | 637 B      | Available   |
| AdventureWorks_Products.csv              | 9/17/2026, 11:02:17 AM | Hot (Inferred) | Block blob | 56.76 KiB  | Available   |
| AdventureWorks>Returns.csv               | 9/17/2026, 11:02:17 AM | Hot (Inferred) | Block blob | 33.78 KiB  | Available   |
| AdventureWorks_Sales_2015.csv            | 9/17/2026, 11:02:18 AM | Hot (Inferred) | Block blob | 115.81 KiB | Available   |
| AdventureWorks_Sales_2016.csv            | 9/17/2026, 11:02:18 AM | Hot (Inferred) | Block blob | 1.03 MiB   | Available   |
| AdventureWorks_Sales_2017.csv            | 9/17/2026, 11:02:19 AM | Hot (Inferred) | Block blob | 1.26 MiB   | Available   |
| AdventureWorks_Territories.csv           | 9/17/2026, 11:02:18 AM | Hot (Inferred) | Block blob | 400 B      | Available   |

Azure Storage — Bronze container with AdventureWorks source files

### Step-by-step explanation

- Open the **bronze** container from the storage account.
- Verify that the raw AdventureWorks CSV files are present, including Calendar, Customers, Product Categories, Product Subcategories, Products, Returns, Sales files, and Territories.
- The Bronze layer acts as the landing area for source data before transformation and modelling.
- Check the filenames and availability of the source files before starting the Databricks notebook.

## Step 3: Configure Databricks Access and Read Calendar Data



Databricks notebook — Spark configuration and Calendar CSV read

### Step-by-step explanation

- Open the Databricks notebook and configure Spark to access the ADLS Gen2 storage account.
- The notebook uses Azure storage authentication settings and the storage endpoint to establish access to the Bronze data.
- Read **AdventureWorks\_Calendar.csv** from the Bronze path using Spark CSV reader options such as **header=true** and **inferSchema=true**.
- Display the DataFrame to confirm that the Calendar data has been loaded successfully.

**Important:** Security note: the screenshot contains credential configuration. For a real project, secrets should be stored in Azure Key Vault / a Databricks secret scope rather than hard-coded in a notebook.

## Step 4: Load and Inspect Customer Data

The screenshot shows a Databricks notebook interface. The top bar indicates 'Microsoft Azure' and 'databricks'. The notebook title is 'New Notebook 2026-09-17 11:13:29'. The left sidebar shows the workspace navigation menu. The main area displays two code cells. The first cell, executed at 12:12 PM (2s), contains the following Python code:

```
df_cust = spark.read.csv(
    "abfss://bronze@kpstorage879.dfs.core.windows.net/AdventureWorks_Customers.csv",
    header=True
)
```

The output of this cell is a Spark DataFrame with 11 fields: CustomerKey, Prefix, First..., LastName, BirthDate, MaritalStatus, Gender, and EmailAddress. The second cell, executed at 12:12 PM (1s), contains the code `df_cust.display()`. The output is a table with 13 rows and 8 columns:

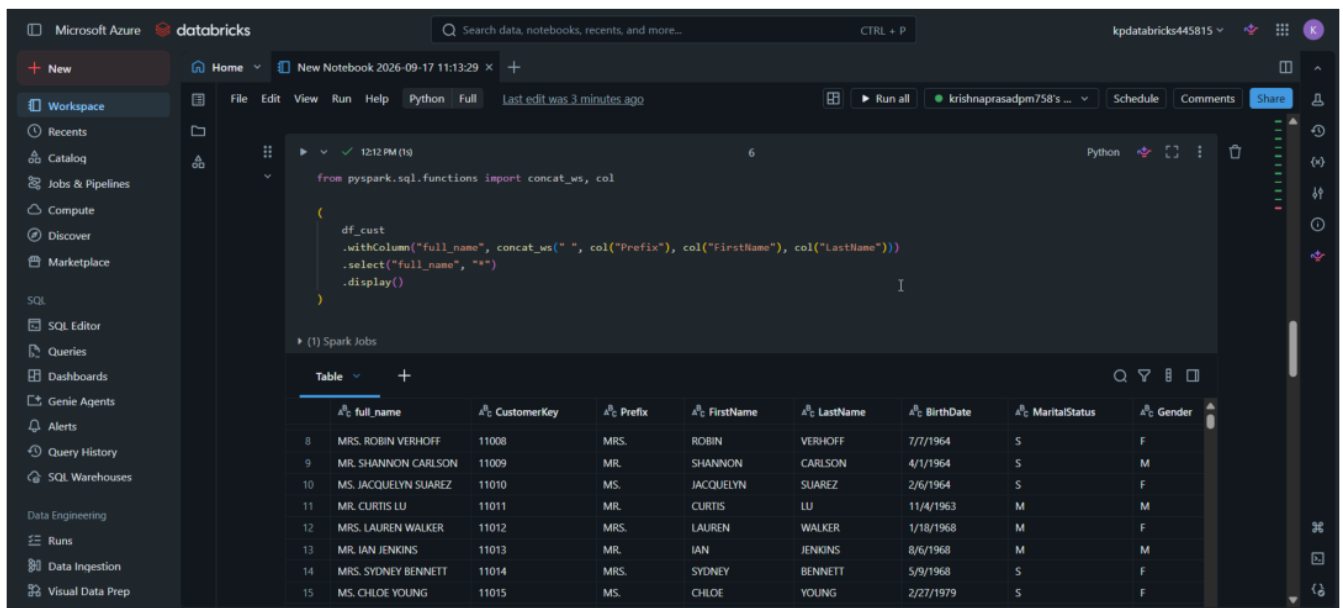
|    | CustomerKey | Prefix | First...  | LastName | BirthDate | MaritalStatus | Gender | EmailAddress           |
|----|-------------|--------|-----------|----------|-----------|---------------|--------|------------------------|
| 7  | 11007       | MR.    | MARCO     | MEHTA    | 5/9/1964  | M             | M      | marco14@adventure-w    |
| 8  | 11008       | MRS.   | ROBIN     | VERHOFF  | 7/7/1964  | S             | F      | rob4@adventure-works   |
| 9  | 11009       | MR.    | SHANNON   | CARLSON  | 4/1/1964  | S             | M      | shannon38@adventure    |
| 10 | 11010       | MS.    | JACQUELYN | SUAREZ   | 2/6/1964  | S             | F      | jacquelyn20@adventure  |
| 11 | 11011       | MR.    | CURTIS    | LU       | 11/4/1963 | M             | M      | curtis9@adventure-work |
| 12 | 11012       | MRS.   | LAUREN    | WALKER   | 1/18/1968 | M             | F      | lauren41@adventure-w   |
| 13 | 11013       | MR.    | IAN       | JENKINS  | 8/6/1968  | M             | M      | ian47@adventure-work   |

Databricks notebook — Reading AdventureWorks Customers

### Step-by-step explanation

- Read **AdventureWorks\_Customers.csv** from the Bronze container with Spark.
- Keep the first row as the header so the source column names are preserved.
- Display the DataFrame to inspect fields such as **CustomerKey**, **Prefix**, **FirstName**, **LastName**, **BirthDate**, **MaritalStatus**, **Gender**, and email information.
- Use this inspection step to understand the raw customer structure before applying transformations.

## Step 5: Standardize Customer Names



```
from pyspark.sql.functions import concat_ws, col

df_cust \
  .withColumn("full_name", concat_ws(" ", col("Prefix"), col("FirstName"), col("LastName"))) \
  .select("full_name", "**") \
  .display()
```

|    | full_name            | CustomerKey | Prefix | FirstName | LastName | BirthDate | MaritalStatus | Gender |
|----|----------------------|-------------|--------|-----------|----------|-----------|---------------|--------|
| 8  | MRS. ROBIN VERHOFF   | 11008       | MRS.   | ROBIN     | VERHOFF  | 7/7/1964  | S             | F      |
| 9  | MR. SHANNON CARLSON  | 11009       | MR.    | SHANNON   | CARLSON  | 4/1/1964  | S             | M      |
| 10 | MS. JACQUELYN SUAREZ | 11010       | MS.    | JACQUELYN | SUAREZ   | 2/6/1964  | S             | F      |
| 11 | MR. CURTIS LU        | 11011       | MR.    | CURTIS    | LU       | 11/4/1963 | M             | M      |
| 12 | MRS. LAUREN WALKER   | 11012       | MRS.   | LAUREN    | WALKER   | 1/18/1968 | M             | F      |
| 13 | MR. IAN JENKINS      | 11013       | MR.    | IAN       | JENKINS  | 8/6/1968  | M             | M      |
| 14 | MRS. SYDNEY BENNETT  | 11014       | MRS.   | SYDNEY    | BENNETT  | 5/9/1968  | S             | F      |
| 15 | MS. CHLOE YOUNG      | 11015       | MS.    | CHLOE     | YOUNG    | 2/27/1979 | S             | F      |

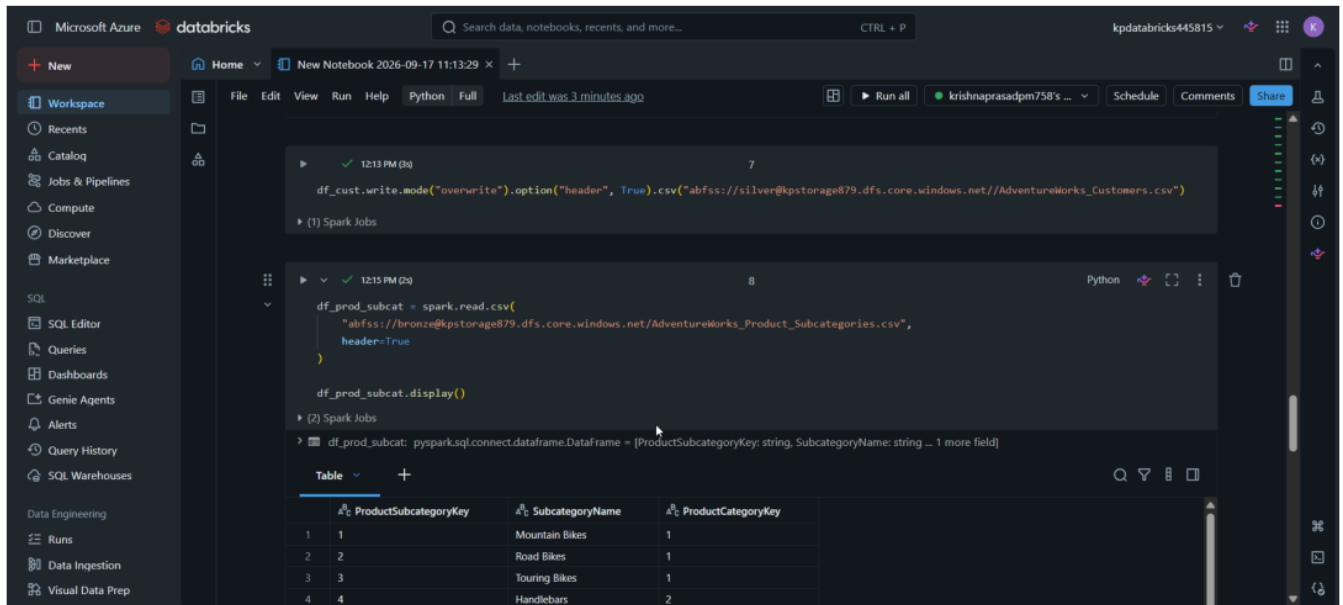
Databricks notebook — Creating a standardized full\_name field

### Step-by-step explanation

- Import the required PySpark SQL functions such as **concat\_ws** and **col**.
- Create a new **full\_name** column by combining **Prefix**, **FirstName**, and **LastName** with a space separator.
- Select the required customer columns together with the new standardized name field.
- Display the result and verify that the transformed customer names are populated correctly.

Azure Data Engineering • Databricks + ADLS Gen2 • Screenshot 5 of 13

## Step 6: Write Customers to Silver and Load Product Subcategories



Databricks notebook — Persisting Customers and reading Product Subcategories

### Step-by-step explanation

- Write the transformed customer DataFrame to the **silver** ADLS container using CSV output.
- Use **overwrite** mode so the target file is refreshed during the demonstrated run.
- Next, read **AdventureWorks\_Product\_Subcategories.csv** from the Bronze container.
- Display the subcategory DataFrame and verify fields such as ProductSubcategoryKey, SubcategoryName, and ProductCategoryKey.

## Step 7: Load and Inspect Product Data

The screenshot shows a Databricks notebook interface. The left sidebar contains navigation options like Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, Data Ingestion, and Visual Data Prep. The main area displays a notebook titled 'New Notebook 2026-09-17 11:13:29'. The code in the notebook is as follows:

```
df_prod = spark.read.csv(
    "abfss://bronze@kpstorage879.dfs.core.windows.net/AdventureWorks_Products.csv",
    header=True
)

df_prod: pyspark.sql.connect.dataframe.DataFrame = [ProductKey: string, ProductSubcategoryKey: string ... 9 more fields]
```

Below the code, the output of `df_prod.display()` is shown as a table with 6 columns: ProductKey, ProductSubcategoryKey, ProductSKU, ProductName, ModelName, and ProductDescription. The table contains 6 rows of data.

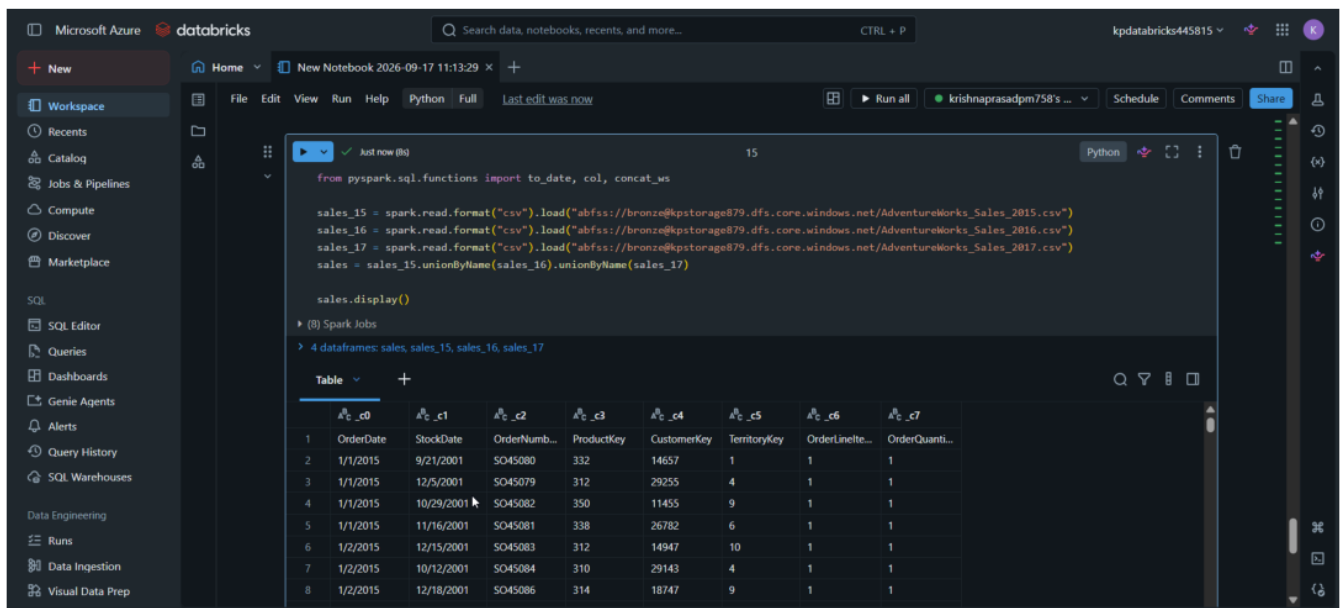
|   | ProductKey | ProductSubcategoryKey | ProductSKU | ProductName             | ModelName           | ProductDescription                       |
|---|------------|-----------------------|------------|-------------------------|---------------------|------------------------------------------|
| 1 | 214        | 31                    | HL-U509-R  | Sport-100 Helmet, Red   | Sport-100           | Universal fit, well-vented, lightweight  |
| 2 | 215        | 31                    | HL-U509    | Sport-100 Helmet, Black | Sport-100           | Universal fit, well-vented, lightweight  |
| 3 | 218        | 23                    | SO-B909-M  | Mountain Bike Socks, M  | Mountain Bike Socks | Combination of natural and synthetic     |
| 4 | 219        | 23                    | SO-B909-L  | Mountain Bike Socks, L  | Mountain Bike Socks | Combination of natural and synthetic     |
| 5 | 220        | 31                    | HL-U509-B  | Sport-100 Helmet, Blue  | Sport-100           | Universal fit, well-vented, lightweight  |
| 6 | 223        | 19                    | CA-1098    | AWC Logo Cap            | Cycling Cap         | Traditional style with a flip-up brim; o |

Databricks notebook — Reading AdventureWorks Products

### Step-by-step explanation

- Read **AdventureWorks\_Products.csv** from the Bronze container using Spark.
- Display the DataFrame to inspect product attributes such as ProductKey, ProductSubcategoryKey, ProductSKU, ProductName, ModelName, and ProductDescription.
- This step prepares the product dataset for subsequent cleaning, modelling, and loading into the Silver layer.
- Confirm that the expected product records and columns are available before continuing.

## Step 8: Combine the Yearly Sales Files



```
from pyspark.sql.functions import to_date, col, concat_ws

sales_15 = spark.read.format("csv").load("abfs://bronze@kpstorage879.dfs.core.windows.net/AdventureWorks_Sales_2015.csv")
sales_16 = spark.read.format("csv").load("abfs://bronze@kpstorage879.dfs.core.windows.net/AdventureWorks_Sales_2016.csv")
sales_17 = spark.read.format("csv").load("abfs://bronze@kpstorage879.dfs.core.windows.net/AdventureWorks_Sales_2017.csv")
sales = sales_15.unionByName(sales_16).unionByName(sales_17)

sales.display()
```

4 dataframes: sales, sales\_15, sales\_16, sales\_17

|   | OrderDate | StockDate  | OrderNum... | ProductKey | CustomerKey | TerritoryKey | OrderLine... | OrderQuanti... |
|---|-----------|------------|-------------|------------|-------------|--------------|--------------|----------------|
| 1 | 1/1/2015  | 9/21/2001  | SO45080     | 332        | 14657       | 1            | 1            |                |
| 2 | 1/1/2015  | 12/5/2001  | SO45079     | 312        | 29255       | 4            | 1            | 1              |
| 3 | 1/1/2015  | 10/29/2001 | SO45082     | 350        | 11455       | 9            | 1            | 1              |
| 4 | 1/1/2015  | 11/16/2001 | SO45081     | 338        | 26782       | 6            | 1            | 1              |
| 5 | 1/2/2015  | 12/15/2001 | SO45083     | 312        | 14947       | 10           | 1            | 1              |
| 6 | 1/2/2015  | 10/12/2001 | SO45084     | 310        | 29143       | 4            | 1            | 1              |
| 7 | 1/2/2015  | 12/18/2001 | SO45086     | 314        | 18747       | 9            | 1            | 1              |
| 8 |           |            |             |            |             |              |              |                |

Databricks notebook — Reading and unioning Sales 2015, 2016 and 2017

### Step-by-step explanation

- Read the three yearly source files: **AdventureWorks\_Sales\_2015.csv**, **AdventureWorks\_Sales\_2016.csv**, and **AdventureWorks\_Sales\_2017.csv**.
- Load each file as a Spark DataFrame using the CSV reader.
- Combine the yearly DataFrames with **unionByName** so that the records form one consolidated Sales dataset.
- Display the combined DataFrame and verify fields such as OrderDate, StockDate, OrderNumber, ProductKey, CustomerKey, TerritoryKey, OrderLineItem, and OrderQuantity.

Azure Data Engineering • Databricks + ADLS Gen2 • Screenshot 8 of 13

## Step 9: Verify the Silver / Processed Layer

Home > kpstorage5646768 | Containers

**silver** Container

Search

+ Add Directory Upload Refresh Delete Copy Paste Rename Acquire lease Break lease Edit columns

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive)

Only show active objects

Showing all 7 items

| <input type="checkbox"/> | Name                  | Last modified         | Access tier | Blob type | Size | Lease state |
|--------------------------|-----------------------|-----------------------|-------------|-----------|------|-------------|
| <input type="checkbox"/> | Calendar.csv          | 9/21/2026, 4:50:41 PM |             |           |      | ...         |
| <input type="checkbox"/> | Customers.csv         | 9/21/2026, 4:50:03 PM |             |           |      | ...         |
| <input type="checkbox"/> | Products.csv          | 9/21/2026, 4:50:14 PM |             |           |      | ...         |
| <input type="checkbox"/> | Returns.csv           | 9/21/2026, 4:50:37 PM |             |           |      | ...         |
| <input type="checkbox"/> | Sales.csv             | 9/21/2026, 4:50:32 PM |             |           |      | ...         |
| <input type="checkbox"/> | Territories.csv       | 9/21/2026, 4:50:46 PM |             |           |      | ...         |
| <input type="checkbox"/> | product_subcategories | 9/21/2026, 4:50:07 PM |             |           |      | ...         |

Add or remove favorites by pressing Ctrl+Shift+F

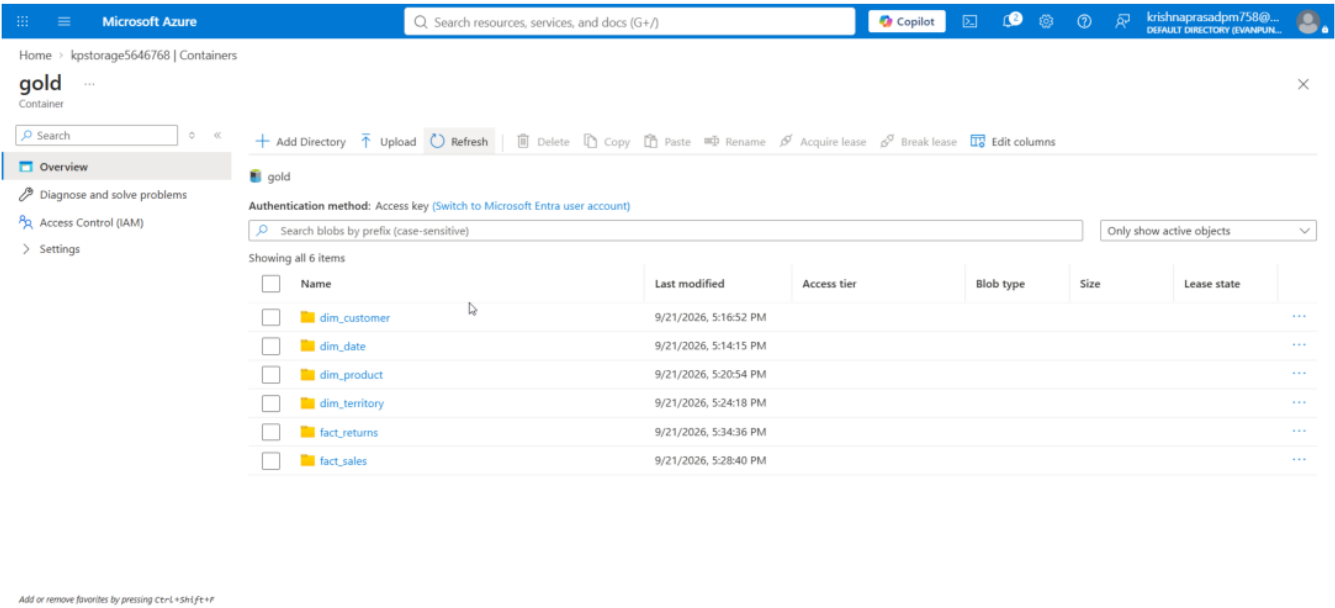
Azure Storage — Silver container

### Step-by-step explanation

- Open the **silver** container after the Databricks processing steps complete.
- Verify the processed datasets: **Calendar.csv**, **Customers.csv**, **Products.csv**, **Returns.csv**, **Sales.csv**, **Territories.csv**, and **product\_subcategories**.
- The presence of these datasets confirms that the raw source files have been processed and persisted into the next data layer.
- Use the Silver layer as the cleaned and standardized foundation for dimensional modelling.

Azure Data Engineering • Databricks + ADLS Gen2 • Screenshot 9 of 13

# Step 10: Verify the Gold / Curated Layer

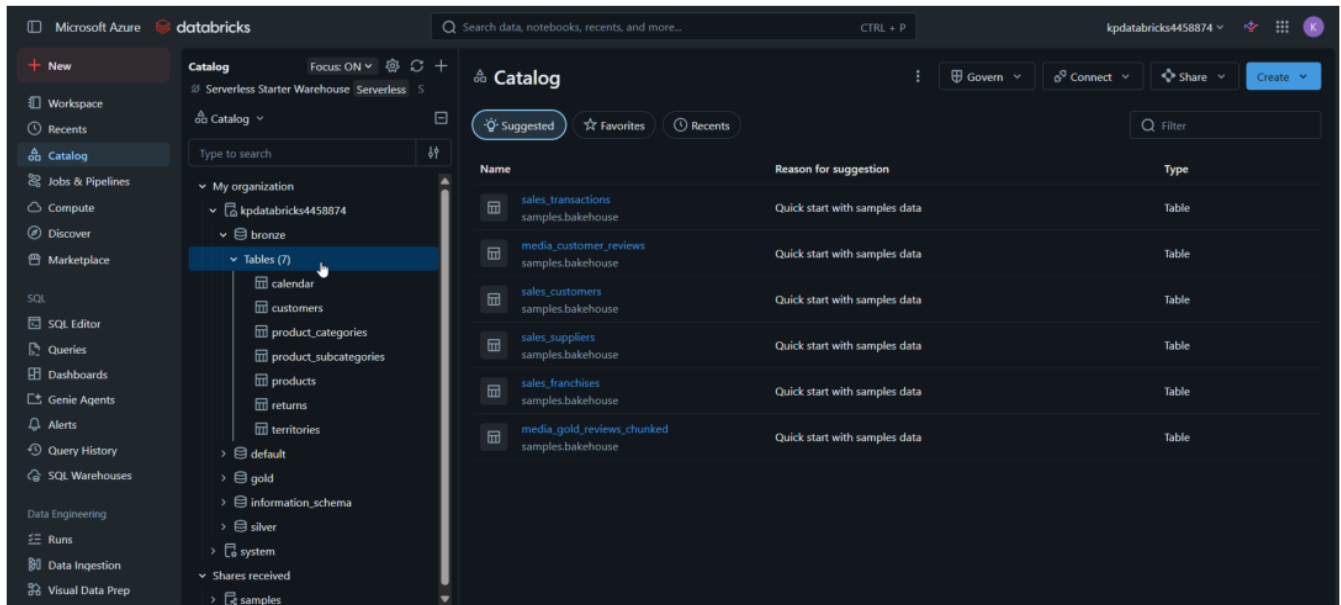


Azure Storage — Gold container

## Step-by-step explanation

- Open the **gold** container to inspect the curated data structures.
- Verify the dimension folders: **dim\_customer**, **dim\_date**, **dim\_product**, and **dim\_territory**.
- Verify the fact folders: **fact\_returns** and **fact\_sales**.
- This structure represents a business-ready dimensional model designed for analytics and reporting.

# Step 11: Register and Inspect the Bronze Layer in Databricks Catalog

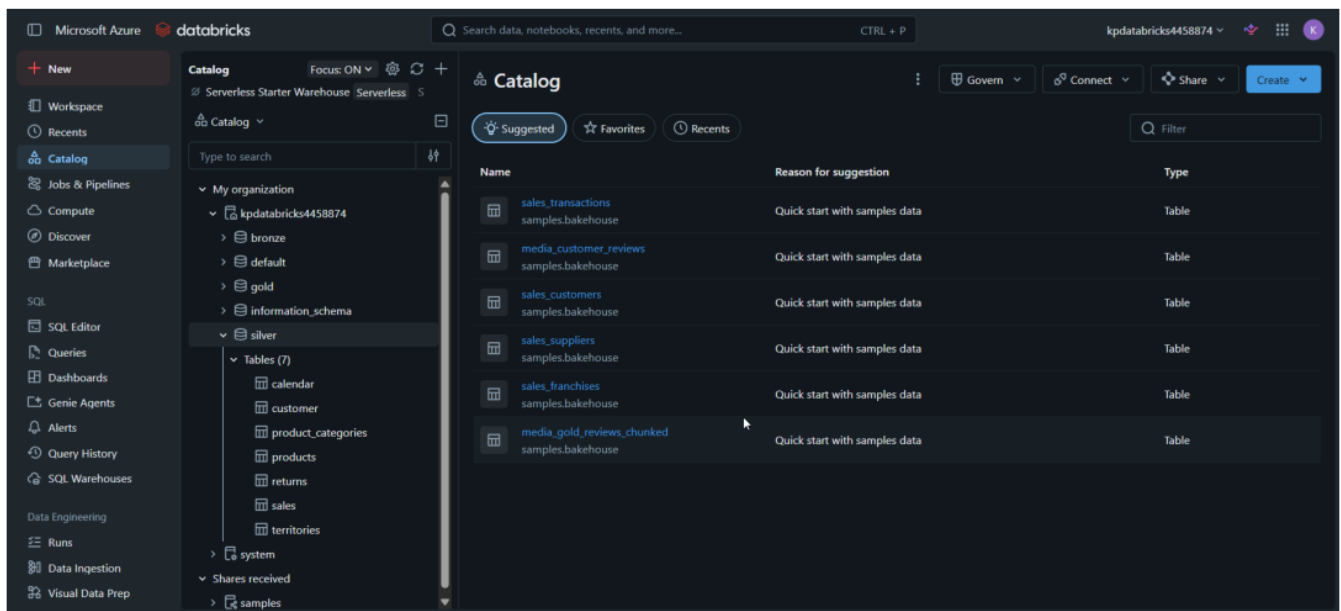


Databricks Catalog — Bronze schema tables

## Step-by-step explanation

- Open **Catalog** in the Databricks workspace and expand the project catalog.
- Select the **bronze** schema and expand its Tables section.
- Verify tables corresponding to the Bronze datasets, including calendar, customers, product\_categories, product\_subcategories, products, returns, and territories.
- Catalog registration makes the data assets discoverable and provides a structured metadata layer for governed analytics.

## Step 12: Inspect the Silver Layer in Databricks Catalog

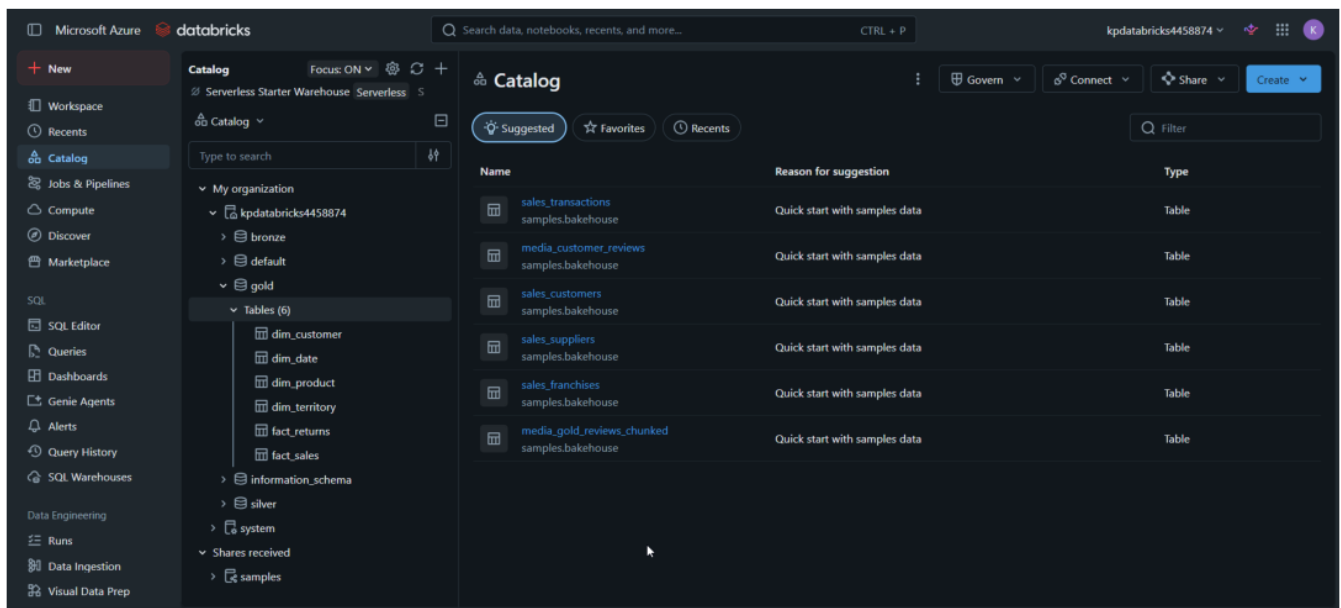


Databricks Catalog — Silver schema tables

### Step-by-step explanation

- Expand the **silver** schema under the project catalog.
- Verify the processed tables: calendar, customer, product\_categories, product\_subcategories, products, returns, sales, and territories.
- The Silver catalog layer provides an organized representation of the cleaned and standardized datasets.
- Use the catalog to locate tables consistently instead of relying only on raw storage paths.

## Step 13: Inspect the Gold Layer in Databricks Catalog



Databricks Catalog — Gold schema tables

### Step-by-step explanation

- Expand the **gold** schema and open its Tables section.
- Verify the curated dimensional model: **dim\_customer**, **dim\_date**, **dim\_product**, **dim\_territory**, **fact\_returns**, and **fact\_sales**.
- The Gold catalog layer exposes analytics-ready dimensions and facts for downstream reporting and BI workloads.
- This completes the demonstrated flow from raw ADLS data through Databricks processing to curated Gold assets.